



UNIVERSIDAD TÉCNICA DE ORURO
FACULTAD NACIONAL DE INGENIERÍA
INGENIERÍA DE SISTEMAS E INFORMÁTICA



**TRABAJO DE CURSO: IMPLEMENTACIÓN
DE ALGORITMOS DE CODIFICACIÓN:
HUFFMAN, FANO Y FANO-SHANNON**

UNIVERSITARIO : HUANCA IRAHOLA FREDDY FERNANDO

DOCENTE : M.SC. ING. FRANZ CHINCHE IMAÑA

AUXILIAR : UNIV. MAMANI ALA MARIANA

ASIGNATURA : TEORIA DE LA INFORMACIÓN

ORURO – BOLIVIA

2025

INDICE

1	OBJETIVO.....	1
1.1	OBJETIVO GENERAL.....	1
1.2	OBJETIVOS ESPECÍFICOS.....	1
2	FUNDAMENTO TEÓRICO	1
2.1	Algoritmo de Shannon-Fano	2
2.2	Algoritmo de Huffman	2
2.3	Algoritmo de Fano	3
2.4	Comparación y Análisis.....	3
3	DISEÑO E IMPLEMENTACIÓN DE LA HERRAMIENTA COMPUTACIONAL	3
4	CONCLUSIONES	6
5	ANEXOS.....	6

1 OBJETIVO

1.1 OBJETIVO GENERAL

Implementar y analizar los algoritmos de codificación Fano, Huffman y Fano-Shannon, con el propósito de comparar su eficiencia en la compresión de información y determinar cual de ellos genera la codificación más optima para un conjunto de datos determinado.

1.2 OBJETIVOS ESPECÍFICOS

- Implementar los algoritmos de Huffman, Fano y Fano-Shannon en un lenguaje de programación.
- Evaluar el rendimiento de cada algoritmo en función de la longitud promedio de código y escoger el mejor código.

2 FUNDAMENTO TEÓRICO

La codificación de información es una técnica fundamental en el campo de las telecomunicaciones y la teoría de la información, cuyo propósito es representar los datos de manera eficiente, reduciendo la redundancia sin perder información. Este proceso permite optimizar el almacenamiento y la transmisión de datos, siendo esencial en sistemas de compresión, criptografía y transmisión digital.

El principio teórico que sustenta estos algoritmos proviene de la Teoría de la Información propuesta por Claude E. Shannon (1948), quien estableció que la cantidad mínima de bits necesarios para representar un símbolo está relacionada con su probabilidad de aparición, según la fórmula:

$$H(X) = - \sum_{i=1}^n p_i \log_2(p_i)$$

donde $H(X)$ es la entropía del conjunto de símbolos, y representa el límite teórico de compresión sin pérdida.

2.1 Algoritmo de Shannon-Fano

El algoritmo de Shannon-Fano fue uno de los primeros métodos de codificación estadística. Su procedimiento consiste en ordenar los símbolos de mayor a menor probabilidad, y luego dividir el conjunto en dos partes con probabilidades totales lo más cercanas posible. A los símbolos del primer grupo se les asigna el bit 0 y a los del segundo el bit 1, repitiendo el proceso de manera recursiva hasta que cada símbolo tenga un código único.

Este método genera códigos prefijos, es decir, ningún código es prefijo de otro, lo que garantiza una decodificación unívoca. Aunque su eficiencia suele ser alta, no siempre alcanza la longitud mínima promedio óptima.

2.2 Algoritmo de Huffman

El algoritmo de Huffman, desarrollado por David A. Huffman en 1952, mejora la eficiencia respecto a Shannon-Fano mediante la construcción de un árbol binario óptimo. El proceso consiste en combinar iterativamente los dos símbolos de menor probabilidad en un nodo padre, sumando sus probabilidades. Este proceso se repite hasta formar un único árbol, asignando 0 a una rama y 1 a la otra.

El código resultante es óptimo en longitud promedio, es decir, ningún otro código de prefijo puede superar su eficiencia para el mismo conjunto de probabilidades. Por esta razón, el algoritmo de Huffman se utiliza ampliamente en compresión de datos sin pérdida, como en los formatos ZIP, JPEG y MP3.

2.3 Algoritmo de Fano

El algoritmo de Fano es una simplificación y adaptación del método Shannon-Fano, en el cual se busca dividir los símbolos en grupos basándose en su frecuencia acumulada, pero utilizando criterios más prácticos en la asignación de bits. Aunque no garantiza la optimalidad de Huffman, ofrece un enfoque más intuitivo y fácil de implementar, siendo útil para comprender los principios básicos de la codificación binaria basada en probabilidades.

2.4 Comparación y Análisis

Los tres algoritmos comparten el objetivo de asignar códigos binarios más cortos a los símbolos más probables, y más largos a los menos probables, pero difieren en su grado de eficiencia y complejidad.

- Huffman logra la codificación más eficiente en la mayoría de los casos.
- Fano-Shannon y Fano son métodos más simples, aunque pueden generar códigos ligeramente óptimos.

La comparación entre ellos permite comprender la evolución de las técnicas de compresión y la relación entre entropía, redundancia y eficiencia en la representación de información.

3 DISEÑO E IMPLEMENTACIÓN DE LA HERRAMIENTA COMPUTACIONAL

La herramienta computacional desarrollada tiene como finalidad la implementación y comparación de tres algoritmos clásicos de codificación: Fano, Fano-Shannon y Huffman. Su propósito es analizar la eficiencia de cada método en función de la entropía y la longitud promedio de los códigos generados, permitiendo determinar cuál ofrece la codificación óptima para un conjunto de símbolos y probabilidades definidos por el usuario.

El programa solicita como entrada:

- El número de símbolos de la fuente (alfabeto).
- Los símbolos que la componen.
- Las probabilidades asociadas a cada símbolo.

Con estos datos, se ejecutan los tres métodos de codificación y se obtienen los códigos correspondientes a cada algoritmo.

Cada módulo cumple la siguiente función:

- Fano: genera combinaciones a partir de una especie de árbol en la que se va generando el código sin importar las probabilidades.
- Fano-Shannon: utiliza una división de longitud de los símbolos y va aumentando código a cada división, procurando que la división sea lo más equivalente posible.
- Huffman: Sigue una tabla de símbolos donde cada par de símbolos de menor valor (o de los que hayan sido ingresados) se suma, quedando solo dos símbolos y se implementa código en cada símbolo y esta va regresando aumentando código a sus antecesores que formaron tal sumatoria.

Finalmente, se implementó un bloque adicional para calcular la entropía del conjunto, la longitud promedio de cada método y su eficiencia porcentual mediante la relación $\eta = (H/L) \times 100$. El sistema identifica automáticamente cuál o cuáles algoritmos presentan la mayor eficiencia, mostrando en pantalla el nombre del método más eficiente y los códigos generados por él.

La herramienta fue desarrollada en el lenguaje Python, debido a su sintaxis clara y a las estructuras de datos facilitan la manipulación de listas y colas.

Se utilizaron las siguientes bibliotecas estándar:

- `collections.deque` para la generación ordenada de códigos en el método Fano.
- `math` para el cálculo de logaritmos y entropía.

El programa opera en consola, permitiendo una interacción directa con el usuario a través de entradas por teclado y resultados impresos en pantalla.

El código conserva una estructura limpia, sin dependencias externas, y puede ser ejecutado en cualquier entorno compatible con Python.

La salida del sistema incluye:

1. Los códigos generados por cada algoritmo.
2. La entropía del conjunto.
3. La longitud promedio y eficiencia de cada método.
4. El o los algoritmos más eficientes, junto con su respectiva codificación.

De esta manera, la herramienta permite no solo observar el funcionamiento interno de cada algoritmo de codificación, sino también comparar de forma cuantitativa su rendimiento y eficiencia, fortaleciendo la comprensión de los principios de compresión de información sin pérdida.

4 CONCLUSIONES

- La implementación de los algoritmos de Fano, Fano-Shannon y Huffman permitió analizar de manera práctica la codificación de fuentes.
- A partir de resultados obtenidos se comprueba que Huffman tiene mayor eficiencia en una mayoría de casos.
- La comparación entre los tres métodos mostró la eficiencia de codificación dependiendo de la distribución de probabilidades.
- El desarrollo de los algoritmos en Python facilitó la implementación para los diferentes conjuntos de datos permitiendo resultados claros, longitud promedio y eficiencia.

5 ANEXOS

Prueba del programa

```
PS D:\Fernando Trabajos\SEMESTRE VI\TEORIA DE LA INFORMACIÓN\Trabajo de curso> p
Ingrese el numero de la fuente: 2
Ingrese el símbolo 1 de la fuente: 0
Ingrese el símbolo 2 de la fuente: 1
Ingrese el numero de símbolos a codificar: 6
Ingrese la probabilidad de S1: 0.05
Ingrese la probabilidad de S2: 0.3
Ingrese la probabilidad de S3: 0.2
Ingrese la probabilidad de S4: 0.2
Ingrese la probabilidad de S5: 0.15
Ingrese la probabilidad de S6: 0.1

FANO
X = ['10', '11', '000', '001', '010', '011']
FANOSHANON
X = ['00', '01', '10', '110', '1110', '1111']
HUFFMAN
X = ['00', '10', '11', '010', '0111', '0110']

--- RESULTADOS DE EFICIENCIA ---
Entropía del conjunto: 2.4087 bits/símbolo
Longitud promedio Fano: 2.6500 -> Eficiencia: 90.89%
Longitud promedio Fano-Shannon: 2.4500 -> Eficiencia: 98.31%
Longitud promedio Huffman: 2.7000 -> Eficiencia: 89.21%

Algoritmo(s) más eficiente(s): Fano-Shannon
Códigos Fano-Shannon: ['00', '01', '10', '110', '1110', '1111']
```